Развој софтвера 2

Подаци, конзистентност и дистрибуиране трансакције

Стефан Капунац

Математички факултет, Универзитет у Београду

школска 2026/27. година

Где смо стали прошли пут?

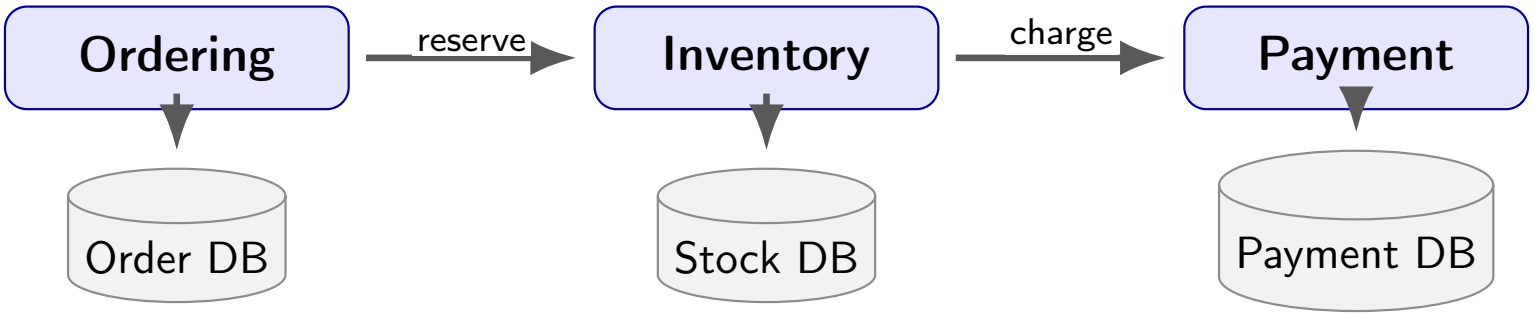
- timeout не значи да операција није извршена
- retry може да направи дупли side effect
- idempotentност и Redis могу да помогну да препознамо исти захтев
- RabbitMQ може поново да испоручи исту поруку

Ново питање

Шта ако једна пословна операција мења стање у више сервиса и више база?

Од локалне до дистрибуиране трансакције

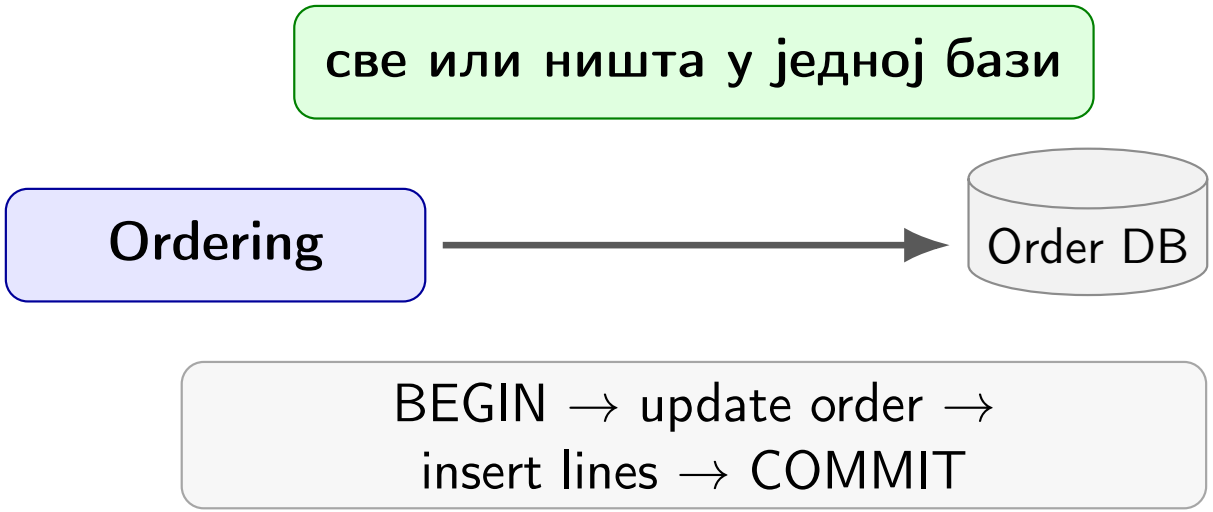
Checkout више није једна трансакција



Желимо један пословни исход

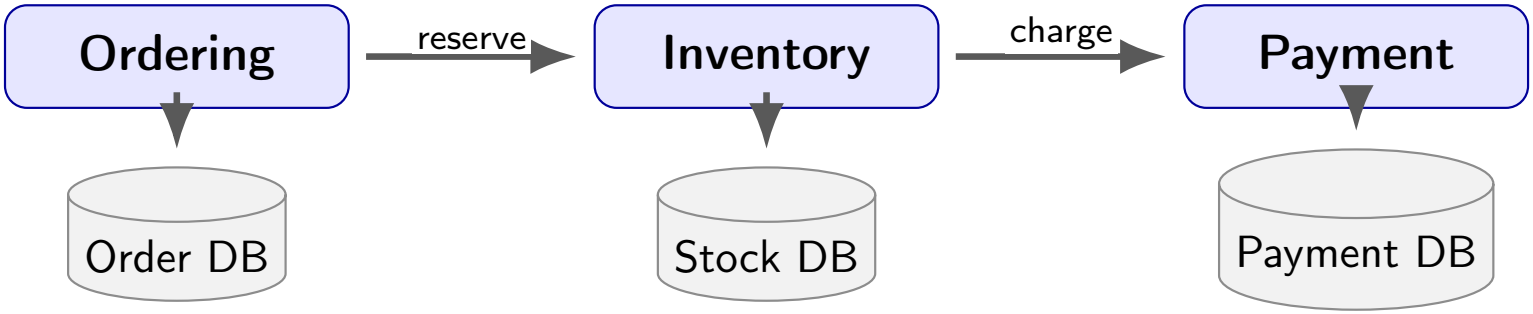
Order је креиран, роба резервисана и плаћање успешно — или систем мора да дође у неко друго конзистентно стање.

Шта нам локална ACID трансакција даје?



- више промена у истој бази можемо да commit-ујемо атомично
- ако нешто не успе, rollback враћа локалну трансакцију
- база контролише concurrency и durability

Али сада имамо три независна commit-а



- Ordering commit-yje order
- Inventory commit-yje reservation
- Payment commit-yje charge

Проблем

Нема једног локалног `ROLLBACK` који може да врати све три базе.

Database per service — цена аутономије

Добро

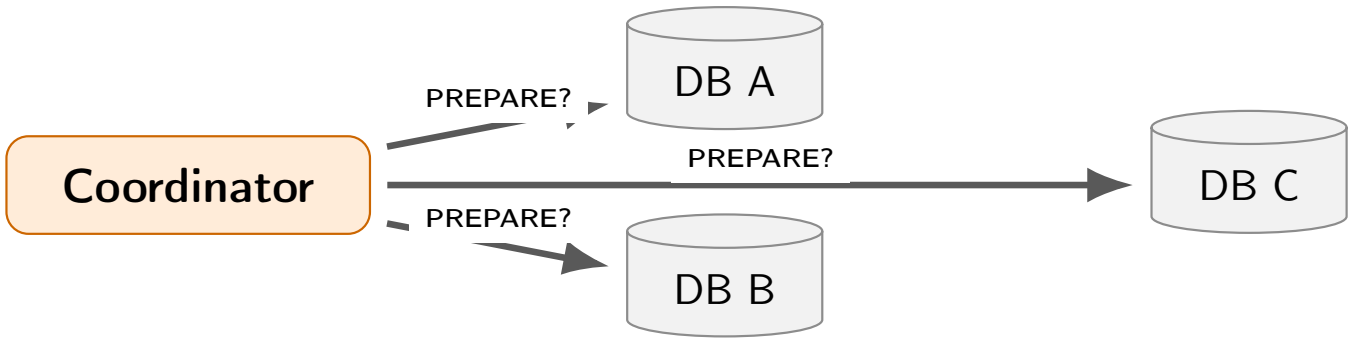
- сервис контролише своју шему
- може самостално да еволуира
- нема директног coupling-а преко табела

Цена

- нема SQL JOIN-а преко сервиса
- нема једне DB трансакције преко свега
- consistency постаје архитектонски проблем

Аутономија није бесплатна.

Прва идеја: distributed transaction / 2PC



ако су сви спремни:
COMMIT; иначе: ROLLBACK

- **prepare**: сваки учесник обећава да може да commit-ује
- **commit/rollback**: coordinator доноси коначну одлуку
- циљ: атомичан исход преко више ресурса

Зашто 2PC није подразумевани одговор за микросервисе?

- учесници морају да подрже coordinated distributed transaction
- ресурси могу дуже остати у prepared/locked стању
- coordinator и recovery постају део failure модела
- external systems (payment provider, email, REST API...) често уопште нису transactional resource

Поента

2PC постоји и важан је концепт. Али у типичној микросервисној архитектури често бирамо локалне трансакције + workflow уместо глобалног commit-а.

Eventual consistency и Saga

Eventual consistency — шта то стварно значи?



Није „подаци су погрешни“

Систем може привремено бити у међустању које је очекивано и моделовано, а затим конвергира ка коначном стању.

Не треба свему иста конзистентност

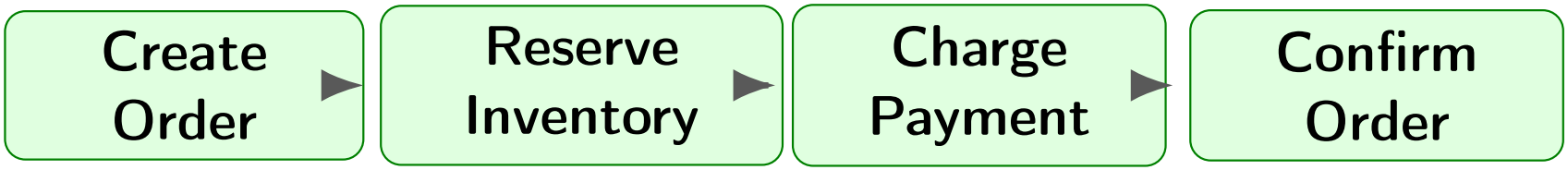
Може да касни

- analytics
- број прегледа
- препоруке
- email нотификација

Критичније

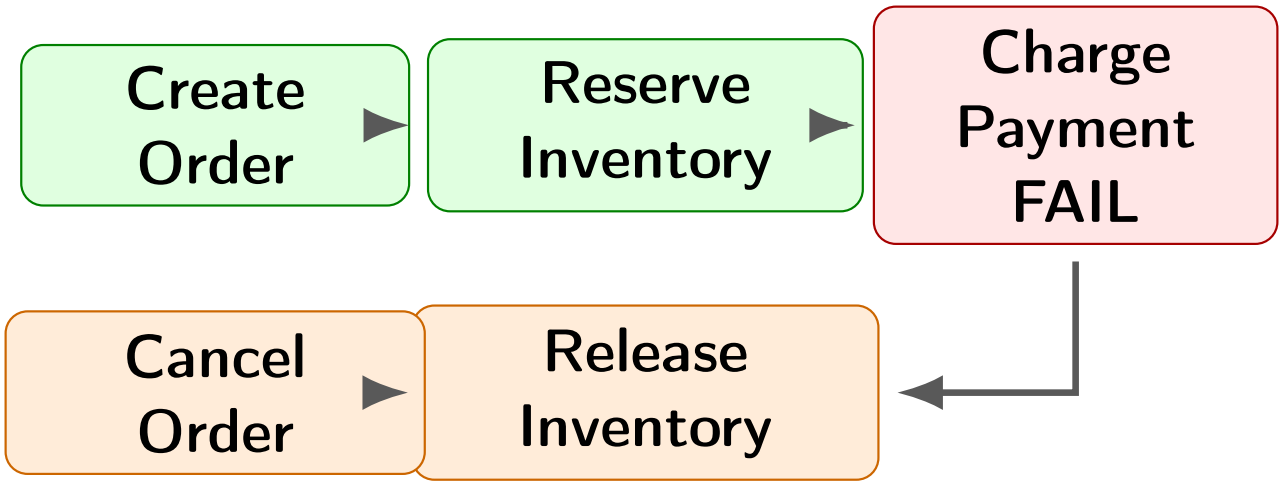
- наплата
- доступна количина
- да ли је order стварно потврђен
- јединственост неких пословних идентификатора

Saga: једна пословна операција, више локалних трансакција



- сваки корак има своју локалну трансакцију
- наредни корак почиње после успешног претходног
- ако нешто не успе, покрећу се compensating actions

Када Payment падне: compensation



Важно

Compensation није магични DB rollback. То је нова пословна операција која покушава да поништи или надокнади претходни ефекат.

Compensation не мора да врати свет у прошлост

Пример

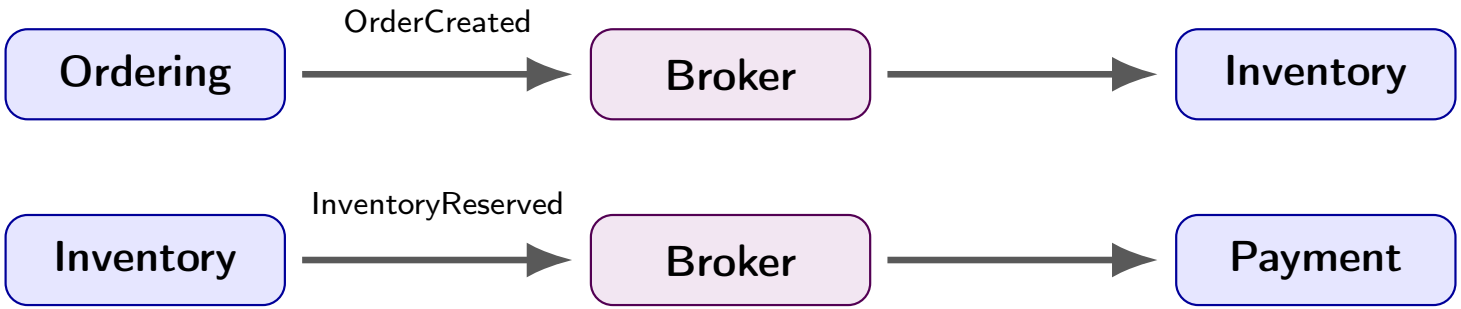
Кориснику је већ послат email „поруџбина је примљена“, а касније Payment не успе.

Не можемо „rollback-овати“ чињеницу да је email виђен.

- можемо послати нови email: „плаћање није успело“
- можемо ослободити резервисану робу
- order може прећи у `PAYMENT_FAILED` или `CANCELLED`

Дизајнирамо исправан следећи state, не time machine.

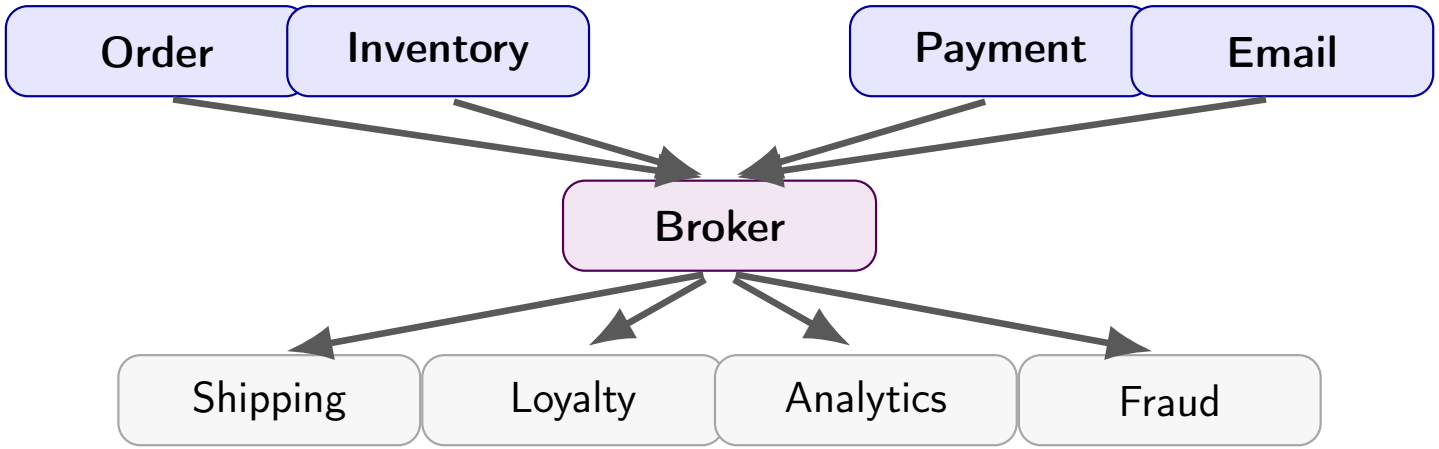
Saga choreography



исти broker и Inventory су поновљени
само да би хронологија била јасна

- нема централног orchestrator-а
- сервиси објављују догађаје и реагују на туђе
- добро за једноставније, loosely-coupled flow-ове

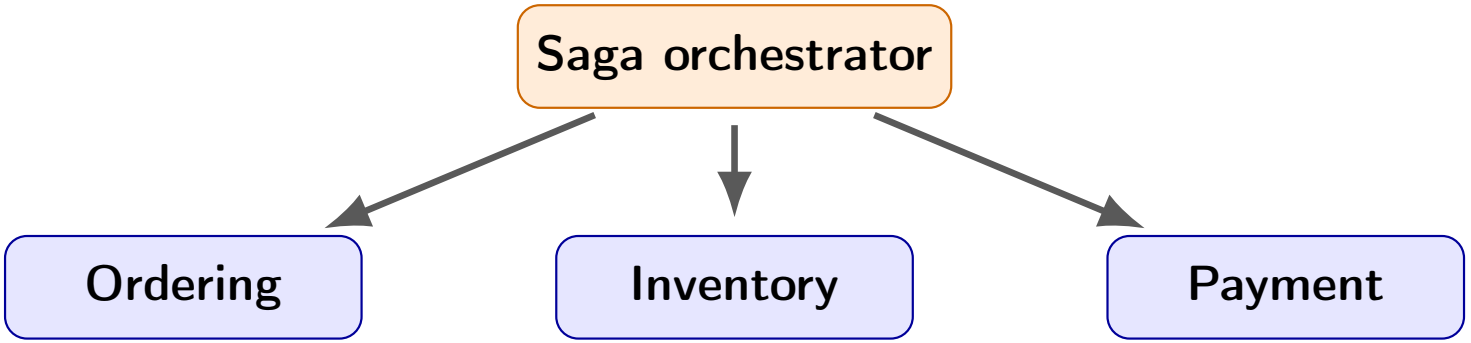
Када choreography постане тешка за праћење



Питање

Ко сада може брзо да одговори: „у којем је кораку checkout и шта се дешава ако Fraud check падне?“

Saga orchestration



координатор експлицитно зна који је
следећи корак и шта треба компензовати

- workflow је експлицитан и лакше се прати
- coordinator зна наредни корак и compensation
- цена: више логике и coupling-a у orchestrator-у

Choreography или orchestration?

Choreography

- мање централне контроле
- природно event-driven
- добро за краће flow-ове
- теже debug-овање сложених зависности

Orchestration

- workflow на једном месту
- лакше видети state machine
- погоднije за сложеније procese
- orchestrator постаје важна компонента

Нема универзално бољег избора.

Мали задатак: шта бисте изабрали?

- После `OrderCreated`: email, analytics и loyalty само реагују независно

Мали задатак: шта бисте изабрали?

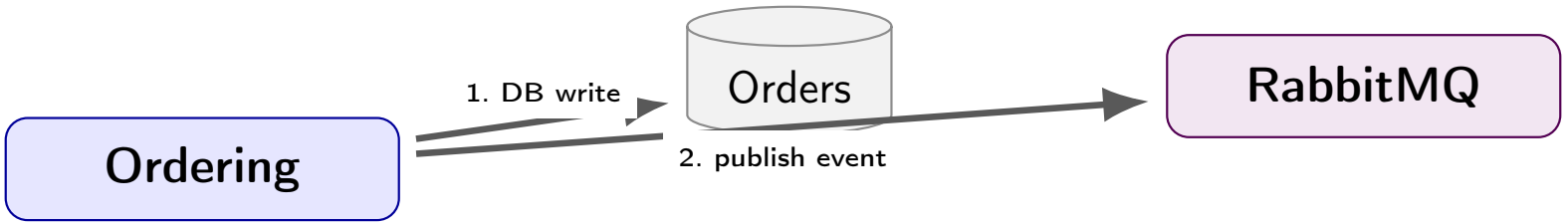
- После `OrderCreated`: email, analytics и loyalty само реагују независно
- Checkout има 8 корака, compensations, timeout-е и ручну интервенцију

Мали задатак: шта бисте изабрали?

- После `OrderCreated`: email, analytics и loyalty само реагују независно
- Checkout има 8 корака, compensations, timeout-е и ручну интервенцију
- Један нови consumer треба да се дода без измене Ordering-а

Dual-write и Transactional Outbox

Још један проблем: DB write + publish event



две независне операције ⇒
нема заједничког COMMIT-а

Желимо

Order постаје `PAID` и објављујемо `OrderPaid`.

Failure #1: DB commit успе, publish не успе

```
1. UPDATE orders SET status='PAID' ...  // COMMIT
2. publish OrderPaid                    // CRASH
```

Последица

У бази је order плаћен, али Inventory/Email/Shipping никада не виде догађај.

Retry publish-a није тривијалан ако смо већ изгубили информацију *шта* је требало послати.

Failure #2: publish успе, DB промена не остане

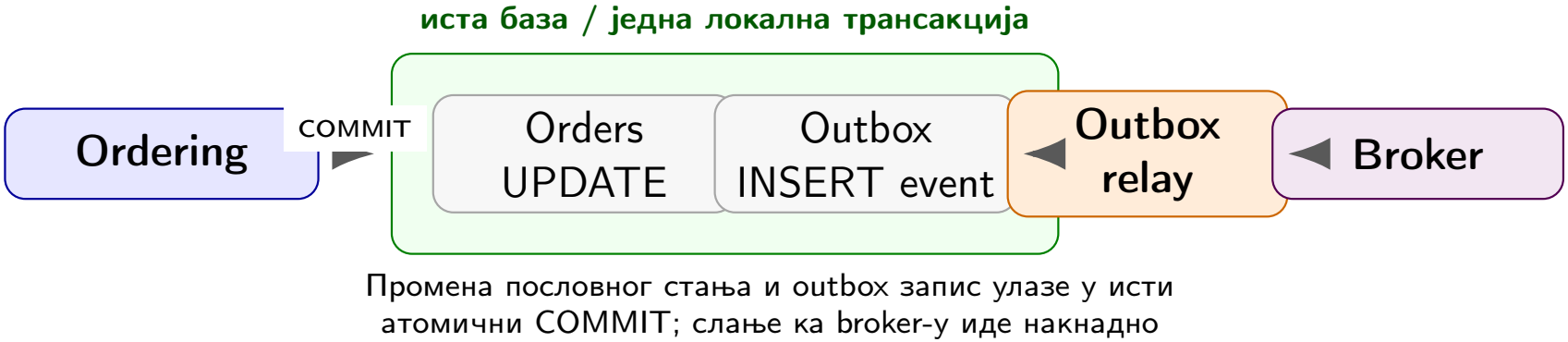
```
1. publish OrderPaid                          // SUCCESS
2. UPDATE orders SET status='PAID' ...  // ROLLBACK / CRASH
```

Последица

Други сервиси виде `OrderPaid`, а Ordering база каже да order није плаћен.

То је dual-write problem.

Transactional Outbox



Промена пословног стања и outbox запис улазе у исти атомични COMMIT; слање ка broker-у иде накнадно

- business промена и outbox запис су у **истој локалној DB трансакцији**
- посебан relay касније шаље outbox запис broker-у
- crash између commit-а и publish-а више не губи догађај — запис остаје у outbox-у

Како outbox изгледа у бази?

```
BEGIN;

UPDATE orders
SET status = 'PAID'
WHERE id = 42;

INSERT INTO outbox(event_id, type, aggregate_id, payload)
VALUES ('evt-991', 'OrderPaid', 42, '{...}');

COMMIT;
```

Кључна ствар

Ако transaction commit-yje — имамо и business state и запис да event мора бити послат.

Ко шаље outbox ка broker-y?

Polling publisher

Процес периодично чита непослате outbox записе и објављује их.

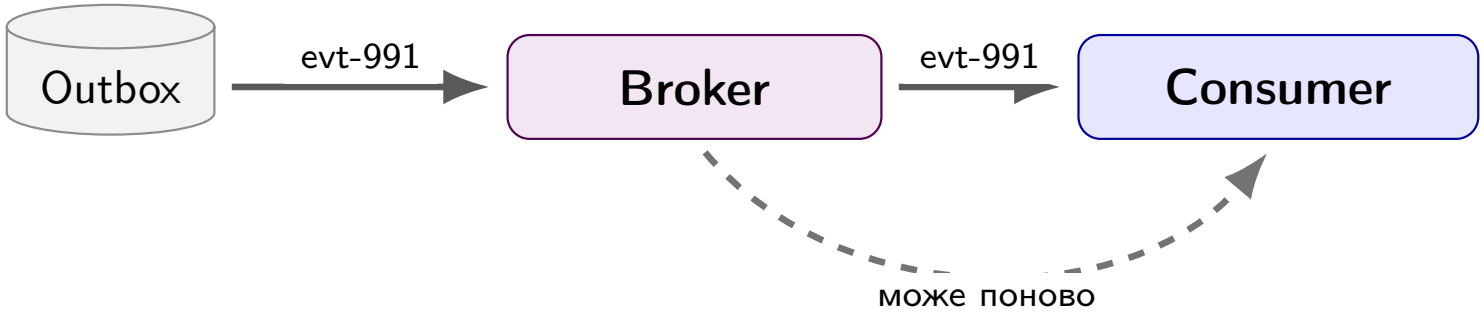
CDC / log-based

Промене се читају из transaction log-a и претварају у event stream.

Оба приступа имају исти важан проблем

Relay може да publish-ује исти event више пута ако падне после слања, а пре него што забележи да је послат.

И ту се враћа прошла недеља: idempotentan consumer

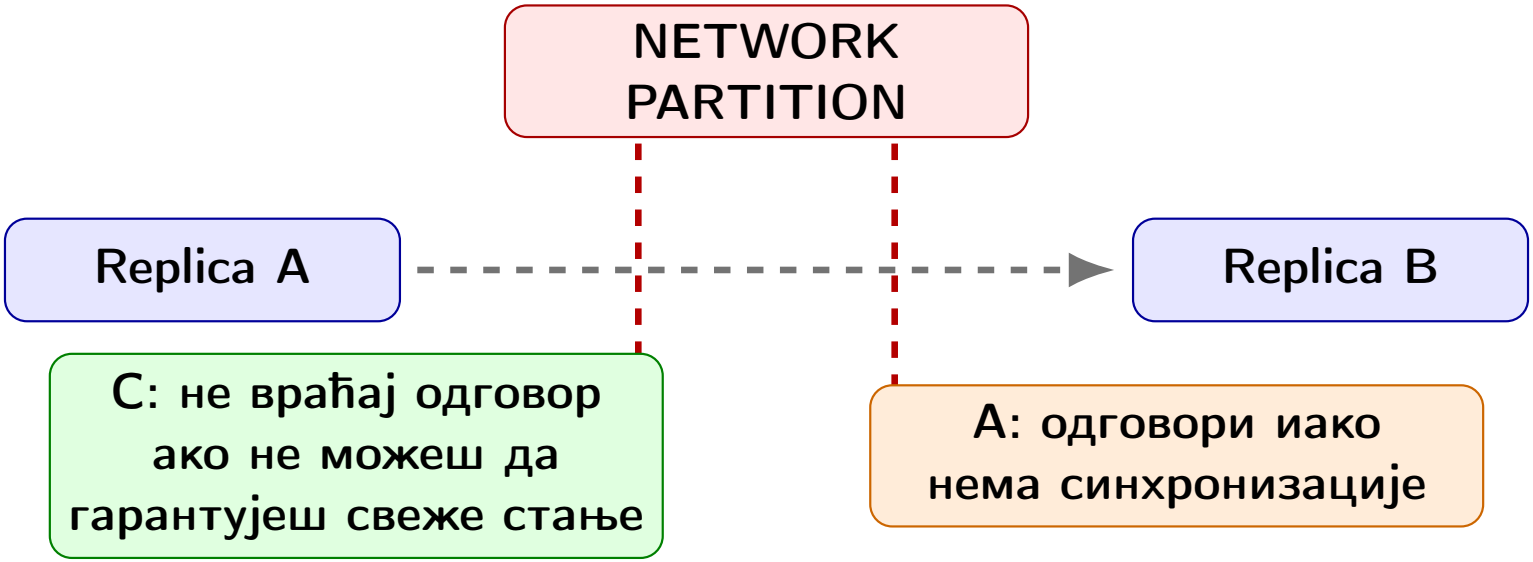


End-to-end поузданост

Outbox спречава губитак event-a; idempotentan consumer спречава дупли business effect.

CAP — укратко

CAP није „изабери било која 2 од 3“



Корисница формулација

Када се деси **network partition**, систем мора да бира између:

- јаке consistency гаранције
- availability за све захтеве

CAP кроз један inventory пример

Две реплике Inventory сервиса у различитим регионима привремено не могу да размене податке.

Изабери consistency

Ако не знамо актуелно стање залиха, одбијамо/чекамо захтев.
Мања доступност, мањи ризик oversell-a.

Изабери availability

Обе стране настављају да прихватају захтеве.
Већа доступност, али могу настати конфликти које касније помиримо.

А када нема partition-а? PACELC интуиција

P: ако има partition-а — consistency или availability?

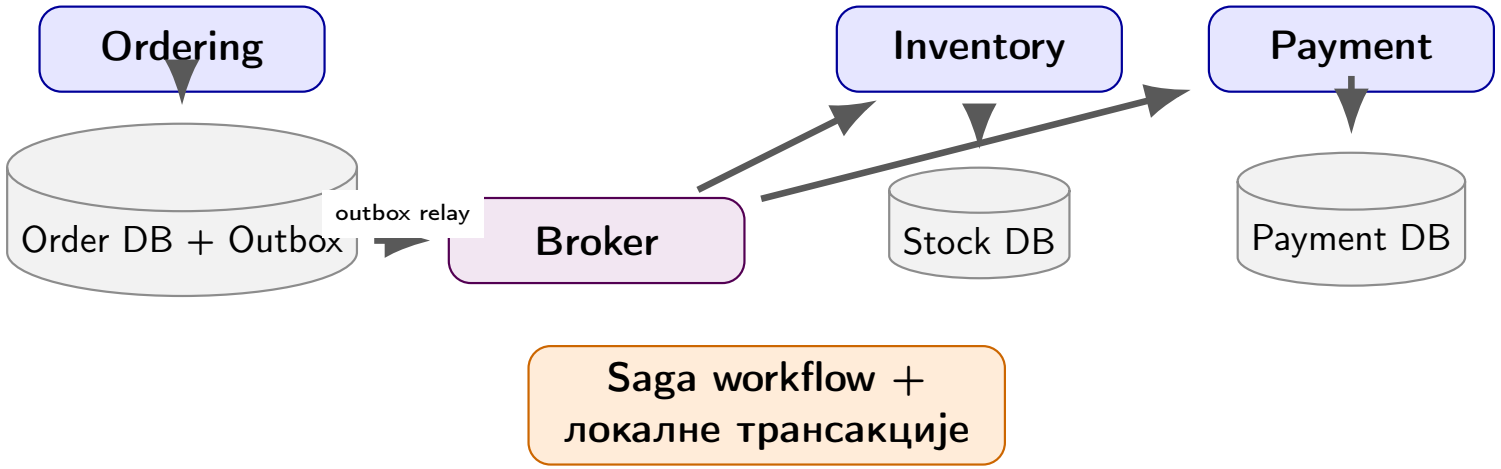
ELSE: ако га нема — latency или consistency?

Не морамо памтити акроним

Поента је да distributed systems trade-off не нестаје када мрежа ради: јаче consistency гаранције често траже додатну координацију и latency.

Спајање свега

Checkout — једно могуће решење



- сваки сервис чува своје податке
- Saga координише пословни ток
- Outbox поуздано повезује DB state и event
- consumers морају бити idempotentni

Како размишљати о проблему?

- Може ли све да стане у **једну локалну трансакцију**? Ако може — најједноставније је најбоље
- Ако не може: да ли нам треба coordinated transaction или **Saga**?
- Koje су валидне **compensation** операције?
- Да ли имамо **dual-write** DB + event?
- Да ли су publisher/consumer дизајнирани за **retry и дупликате**?

Шта треба запамтити

- ACID одлично решава локалну трансакциону границу — не цео distributed workflow
- Eventual consistency треба да буде **моделована**, не случајна
- Saga = низ локалних трансакција + compensation када је потребна
- Choreography и orchestration су два начина координације Saga workflow-a
- Transactional Outbox решава DB + publish dual-write проблем
- At-least-once delivery значи да idempotentnost остаје важна
- CAP говори о избору C/A **када partition стварно постоји**

За даље читање

- Microsoft Azure Architecture Center — *Saga pattern*
- AWS Prescriptive Guidance — *Transactional outbox pattern*
- PostgreSQL documentation — *Two-Phase Transactions*
- Gilbert & Lynch — *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*